```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau


tf.random.set_seed(42)
np.random.seed(42)

train_df=pd.read_csv("train.csv")
train_df.head()
```

```
{"type":"dataframe","variable_name":"train_df"}
```

```
x=train_df.iloc[:,:-1].values.reshape(-1,3,32,32).transpose(0,2,3,1)
y=train_df.iloc[:,-1].values.astype("int64")

xtrain,xtest, ytrain,ytest=train_test_split(x,y,test_size=0.2,
random_state=42)

print("Train", xtrain.shape,ytrain.shape)
print("test", xtest.shape, ytest.shape)
print("classes" ,np.unique(ytrain))
```

```
Train (40000, 32, 32, 3) (40000,)
test (10000, 32, 32, 3) (10000,)
classes [0 1 2 3 4 5 6 7 8 9]
```

```
plt.figure(figsize=(12, 8))
for i in range(10):
  plt.subplot(1,10,i+1)
  plt.imshow(xtrain[i], interpolation="nearest")
  plt.title(ytrain[i])
  plt.axis("off")
plt.tight_layout()
plt.show()
```



```
xtrain=xtrain.astype("float32")/255.0
xtest=xtest.astype("float32")/255.0

ytrain_sc=to_categorical(ytrain,10)
ytest_sc=to_categorical(ytest,10)

model=keras.Sequential([
    layers.Input(shape=(32,32,3)),
    layers.Flatten(),
    layers.Dense(1024, activation="relu"),
    layers.BatchNormalization(),
    layers.Dropout(0.2),
    layers.Dense(512, activation="relu"),
    layers.BatchNormalization(),
    layers.Dropout(0.2),
    layers.Dense(256,activation="relu"),
    layers.BatchNormalization(),
    layers.Dropout(0.2),
    layers.Dense(10,activation="softmax"),

])
model.summary()
```

```
Model: "sequential_3"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃         Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
│ flatten_3 (Flatten)                  │ (None, 3072)                │               0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dense_10 (Dense)                     │ (None, 1024)                │       3,146,752 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ batch_normalization_5                │ (None, 1024)                │           4,096 │
│ (BatchNormalization)                 │                             │                 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dropout_5 (Dropout)                  │ (None, 1024)                │               0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dense_11 (Dense)                     │ (None, 512)                 │
```

```
524,800 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ batch_normalization_6           │ (None, 512)            │
2,048 │
│ (BatchNormalization)            │                        │
      │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dropout_6 (Dropout)             │ (None, 512)            │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dense_12 (Dense)                │ (None, 256)            │
131,328 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ batch_normalization_7           │ (None, 256)            │
1,024 │
│ (BatchNormalization)            │                        │
      │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dropout_7 (Dropout)             │ (None, 256)            │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dense_13 (Dense)                │ (None, 10)             │
2,570 │
└─────────────────────────────────┴────────────────────────┴──────────
──────┘

 Total params: 3,812,618 (14.54 MB)

 Trainable params: 3,809,034 (14.53 MB)

 Non-trainable params: 3,584 (14.00 KB)
```

```
model.compile(
    loss="categorical_crossentropy",
    optimizer=Adam(learning_rate=0.0005),
    metrics=["accuracy"]
)
callbacks = [EarlyStopping(monitor="val_loss", patience=8,
restore_best_weights=True),
             ReduceLROnPlateau(monitor="val_loss", factor=0.5,
patience=3)]
history=model.fit(
    xtrain,
    ytrain_sc,
```

```
    validation_split=0.1,
    epochs=40,
    batch_size=64,
    verbose=2,
    callbacks=callbacks
)

Epoch 1/40
2250/2250 - 16s - 7ms/step - accuracy: 0.4471 - loss: 1.5695 -
val_accuracy: 0.4300 - val_loss: 1.6195 - learning_rate: 5.0000e-04
Epoch 2/40
2250/2250 - 9s - 4ms/step - accuracy: 0.4285 - loss: 1.6026 -
val_accuracy: 0.4092 - val_loss: 1.6341 - learning_rate: 5.0000e-04
Epoch 3/40
2250/2250 - 9s - 4ms/step - accuracy: 0.4264 - loss: 1.6124 -
val_accuracy: 0.4532 - val_loss: 1.5403 - learning_rate: 5.0000e-04
Epoch 4/40
2250/2250 - 9s - 4ms/step - accuracy: 0.4341 - loss: 1.5835 -
val_accuracy: 0.4200 - val_loss: 1.6132 - learning_rate: 5.0000e-04
Epoch 5/40
2250/2250 - 8s - 4ms/step - accuracy: 0.4421 - loss: 1.5656 -
val_accuracy: 0.4428 - val_loss: 1.5790 - learning_rate: 5.0000e-04
Epoch 6/40
2250/2250 - 9s - 4ms/step - accuracy: 0.4519 - loss: 1.5404 -
val_accuracy: 0.4625 - val_loss: 1.5143 - learning_rate: 5.0000e-04
Epoch 7/40
2250/2250 - 9s - 4ms/step - accuracy: 0.4518 - loss: 1.5406 -
val_accuracy: 0.4600 - val_loss: 1.4959 - learning_rate: 5.0000e-04
Epoch 8/40
2250/2250 - 8s - 4ms/step - accuracy: 0.4542 - loss: 1.5300 -
val_accuracy: 0.4767 - val_loss: 1.4796 - learning_rate: 5.0000e-04
Epoch 9/40
2250/2250 - 9s - 4ms/step - accuracy: 0.4573 - loss: 1.5216 -
val_accuracy: 0.4825 - val_loss: 1.4524 - learning_rate: 5.0000e-04
Epoch 10/40
2250/2250 - 9s - 4ms/step - accuracy: 0.4624 - loss: 1.5076 -
val_accuracy: 0.4782 - val_loss: 1.4419 - learning_rate: 5.0000e-04
Epoch 11/40
2250/2250 - 8s - 4ms/step - accuracy: 0.4618 - loss: 1.5096 -
val_accuracy: 0.4708 - val_loss: 1.5356 - learning_rate: 5.0000e-04
Epoch 12/40
2250/2250 - 9s - 4ms/step - accuracy: 0.4645 - loss: 1.4994 -
val_accuracy: 0.4730 - val_loss: 1.4619 - learning_rate: 5.0000e-04
Epoch 13/40
2250/2250 - 8s - 4ms/step - accuracy: 0.4771 - loss: 1.4741 -
val_accuracy: 0.4712 - val_loss: 1.4806 - learning_rate: 5.0000e-04
Epoch 14/40
2250/2250 - 9s - 4ms/step - accuracy: 0.4929 - loss: 1.4251 -
val_accuracy: 0.5070 - val_loss: 1.3903 - learning_rate: 2.5000e-04
Epoch 15/40
```

```
2250/2250 - 9s - 4ms/step - accuracy: 0.5001 - loss: 1.4050 -
val_accuracy: 0.4997 - val_loss: 1.4345 - learning_rate: 2.5000e-04
Epoch 16/40
2250/2250 - 8s - 4ms/step - accuracy: 0.5012 - loss: 1.4016 -
val_accuracy: 0.5020 - val_loss: 1.4009 - learning_rate: 2.5000e-04
Epoch 17/40
2250/2250 - 9s - 4ms/step - accuracy: 0.4999 - loss: 1.4027 -
val_accuracy: 0.4925 - val_loss: 1.4286 - learning_rate: 2.5000e-04
Epoch 18/40
2250/2250 - 9s - 4ms/step - accuracy: 0.5159 - loss: 1.3597 -
val_accuracy: 0.5117 - val_loss: 1.3937 - learning_rate: 1.2500e-04
Epoch 19/40
2250/2250 - 8s - 4ms/step - accuracy: 0.5214 - loss: 1.3496 -
val_accuracy: 0.5150 - val_loss: 1.3963 - learning_rate: 1.2500e-04
Epoch 20/40
2250/2250 - 9s - 4ms/step - accuracy: 0.5224 - loss: 1.3434 -
val_accuracy: 0.5142 - val_loss: 1.3798 - learning_rate: 1.2500e-04
Epoch 21/40
2250/2250 - 9s - 4ms/step - accuracy: 0.5265 - loss: 1.3335 -
val_accuracy: 0.5173 - val_loss: 1.4037 - learning_rate: 1.2500e-04
Epoch 22/40
2250/2250 - 8s - 4ms/step - accuracy: 0.5282 - loss: 1.3317 -
val_accuracy: 0.5163 - val_loss: 1.4063 - learning_rate: 1.2500e-04
Epoch 23/40
2250/2250 - 9s - 4ms/step - accuracy: 0.5288 - loss: 1.3274 -
val_accuracy: 0.5190 - val_loss: 1.3897 - learning_rate: 1.2500e-04
Epoch 24/40
2250/2250 - 10s - 4ms/step - accuracy: 0.5371 - loss: 1.3085 -
val_accuracy: 0.5197 - val_loss: 1.3767 - learning_rate: 6.2500e-05
Epoch 25/40
2250/2250 - 8s - 4ms/step - accuracy: 0.5378 - loss: 1.3050 -
val_accuracy: 0.5165 - val_loss: 1.3762 - learning_rate: 6.2500e-05
Epoch 26/40
2250/2250 - 9s - 4ms/step - accuracy: 0.5379 - loss: 1.3003 -
val_accuracy: 0.5242 - val_loss: 1.3717 - learning_rate: 6.2500e-05
Epoch 27/40
2250/2250 - 8s - 4ms/step - accuracy: 0.5382 - loss: 1.2971 -
val_accuracy: 0.5142 - val_loss: 1.3768 - learning_rate: 6.2500e-05
Epoch 28/40
2250/2250 - 9s - 4ms/step - accuracy: 0.5403 - loss: 1.2992 -
val_accuracy: 0.5230 - val_loss: 1.3739 - learning_rate: 6.2500e-05
Epoch 29/40
2250/2250 - 9s - 4ms/step - accuracy: 0.5387 - loss: 1.2903 -
val_accuracy: 0.5213 - val_loss: 1.3843 - learning_rate: 6.2500e-05
Epoch 30/40
2250/2250 - 8s - 4ms/step - accuracy: 0.5406 - loss: 1.2888 -
val_accuracy: 0.5213 - val_loss: 1.3809 - learning_rate: 3.1250e-05
Epoch 31/40
2250/2250 - 9s - 4ms/step - accuracy: 0.5421 - loss: 1.2833 -
```

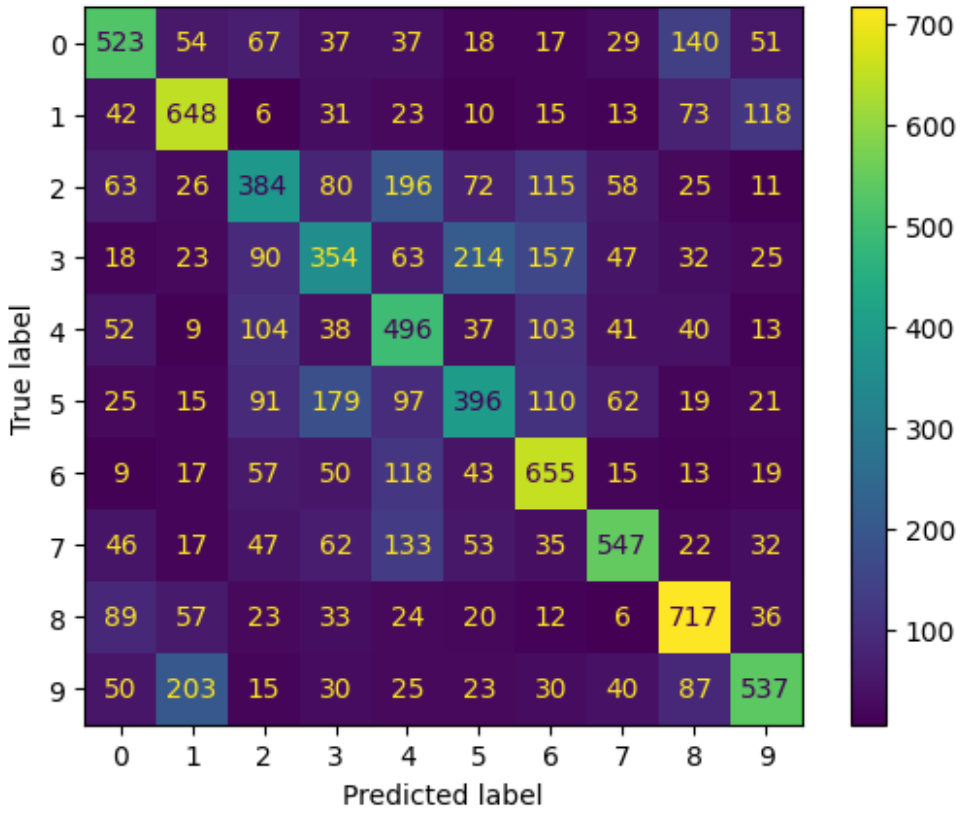
```
val_accuracy: 0.5215 - val_loss: 1.3845 - learning_rate: 3.1250e-05
Epoch 32/40
2250/2250 - 9s - 4ms/step - accuracy: 0.5460 - loss: 1.2804 -
val_accuracy: 0.5228 - val_loss: 1.3729 - learning_rate: 3.1250e-05
Epoch 33/40
2250/2250 - 8s - 4ms/step - accuracy: 0.5450 - loss: 1.2771 -
val_accuracy: 0.5253 - val_loss: 1.3755 - learning_rate: 1.5625e-05
Epoch 34/40
2250/2250 - 9s - 4ms/step - accuracy: 0.5430 - loss: 1.2796 -
val_accuracy: 0.5263 - val_loss: 1.3768 - learning_rate: 1.5625e-05
```

```
test_loss, test_acc=model.evaluate(xtest,ytest_sc)
print("test loss", test_loss)
print("test acc", test_acc)

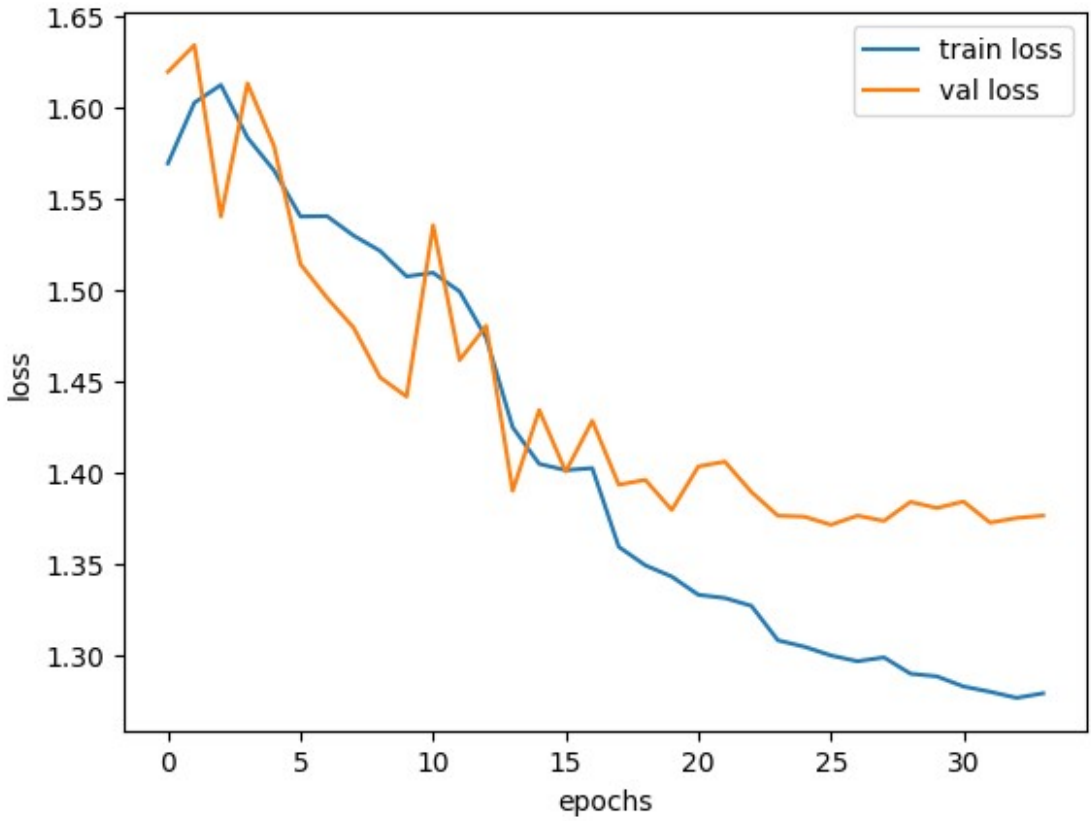
ypred=model.predict(xtest).argmax(axis=1)

print(confusion_matrix(ytest,ypred))
ConfusionMatrixDisplay.from_predictions(ytest,ypred)
plt.show()
```

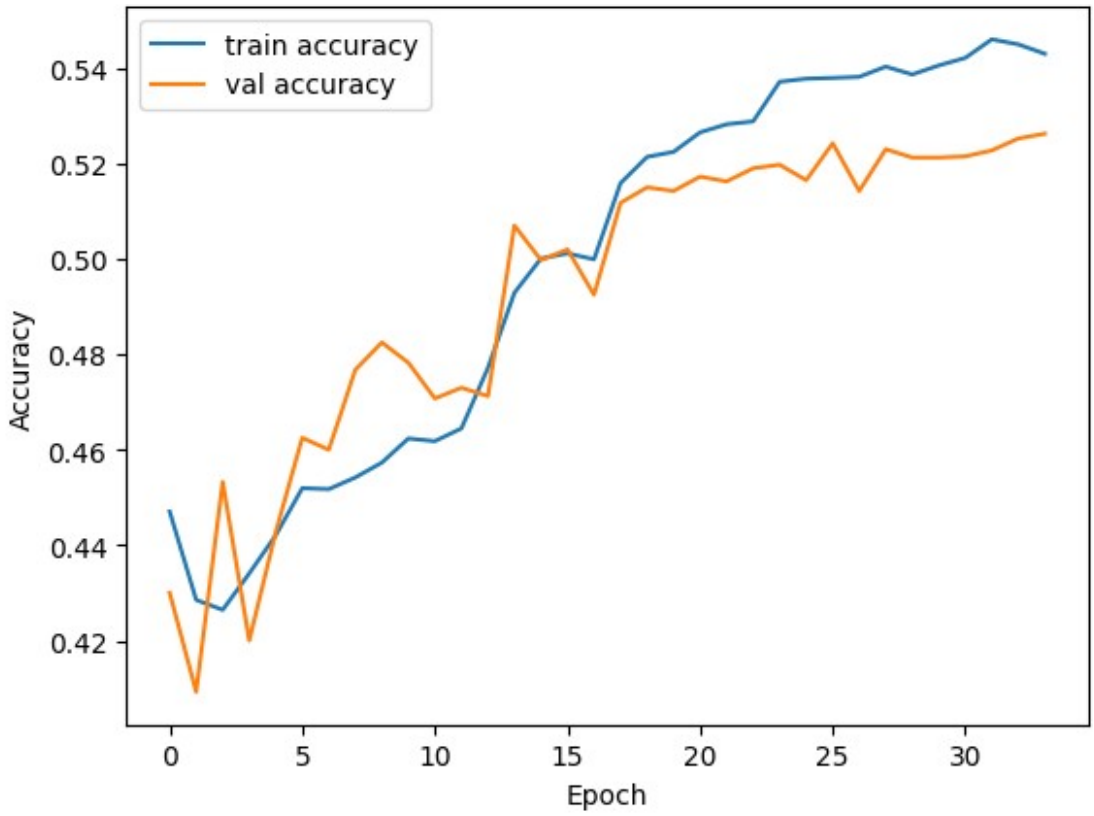
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.5257 - loss:
1.3592
test loss 1.3592274188995361
test acc 0.5256999731063843
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
[[523  54  67  37  37  18  17  29 140  51]
 [ 42 648   6  31  23  10  15  13  73 118]
 [ 63  26 384  80 196  72 115  58  25  11]
 [ 18  23  90 354  63 214 157  47  32  25]
 [ 52   9 104  38 496  37 103  41  40  13]
 [ 25  15  91 179  97 396 110  62  19  21]
 [  9  17  57  50 118  43 655  15  13  19]
 [ 46  17  47  62 133  53  35 547  22  32]
 [ 89  57  23  33  24  20  12   6 717  36]
 [ 50 203  15  30  25  23  30  40  87 537]]
```




```
plt.plot(history.history["loss"],label="train loss")
plt.plot(history.history["val_loss"], label="val loss")
plt.xlabel("epochs")
plt.ylabel("loss")
plt.legend()
plt.show()
```




```
plt.plot(history.history["accuracy"], label="train accuracy")
plt.plot(history.history["val_accuracy"], label="val accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```


train accuracy
val accuracy
0.54
0.52
0.50
0.48
0.46
0.44
0.42
Accuracy
0
5
10
15
20
25
30
Epoch